

Experiment 11

Data Driven Login Testing with Python Behave

This experiment demonstrates how a single Behave Scenario Outline can validate several login outcomes by reading credentials and expected results from an Examples table.

Experiment Overview

Module	Python BDD RESTful Automations
Objective	Automate login validation with the Python Behave framework.
Core idea	Replace repeated Gherkin scenarios with one parameterized Scenario Outline.
Tools	Python 3, Behave, Selenium WebDriver, Gherkin, and an Examples table.

Problem and Approach

Writing an individual scenario for every valid, invalid, or locked account makes a feature file repetitive and difficult to extend. A Scenario Outline solves this by keeping the behavior in one place and supplying different inputs through a table. Behave runs the outline once for each row.

Feature Specification

The feature opens the Sauce Demo login page, enters one row of credentials, submits the form, and checks the expected outcome. The three test rows are:

Username	Password	Expected result
standard_user	secret_sauce	success
locked_out_user	secret_sauce	locked
invalid_user	wrong_pass	failure

Implementation Details

The step definitions create a headless Chrome session before each scenario, navigate to the login page, fill the username and password fields, click the login button, and then assert the result. A successful login must reach inventory.html. The two negative cases must display the corresponding error message. The browser is closed after every scenario so that each example starts in a clean state.

Project Structure

- features/data_driven_login.feature - Scenario Outline and Examples table
- features/environment.py - Browser setup and cleanup hooks
- features/steps/data_driven_steps.py - Given, When, and Then definitions
- requirements.txt - Python dependencies

Execution in VS Code

Open the project folder in VS Code and run the following commands in the integrated terminal:

```
python -m venv .venv

.\.venv\Scripts\Activate.ps1

python -m pip install -r requirements.txt

python -m behave features/data_driven_login.feature
```

```
Feature: Data-driven login validation # features/data_driven_login.feature:1

  Scenario Outline: Validate login behavior for multiple credential sets -- @1.1 # features/data_driven_login.feature:10
    Given the user opens the login portal # features/steps/data_driven_steps.py:5
    When the user enters username "standard_user" and password "secret_sauce" # features/steps/data_driven_steps.py:10
    And submits the login form # features/steps/data_driven_steps.py:16
    Then the login result should be "success" # features/steps/data_driven_steps.py:21

  Scenario Outline: Validate login behavior for multiple credential sets -- @1.2 # features/data_driven_login.feature:11
    Given the user opens the login portal # features/steps/data_driven_steps.py:5
    When the user enters username "locked_out_user" and password "secret_sauce" # features/steps/data_driven_steps.py:10
    And submits the login form # features/steps/data_driven_steps.py:16
    Then the login result should be "locked" # features/steps/data_driven_steps.py:21

  Scenario Outline: Validate login behavior for multiple credential sets -- @1.3 # features/data_driven_login.feature:12
    Given the user opens the login portal # features/steps/data_driven_steps.py:5
    When the user enters username "invalid_user" and password "wrong_pass" # features/steps/data_driven_steps.py:10
    And submits the login form # features/steps/data_driven_steps.py:16
    Then the login result should be "failure" # features/steps/data_driven_steps.py:21

1 feature passed, 0 failed, 0 skipped
3 scenarios passed, 0 failed, 0 skipped
12 steps passed, 0 failed, 0 skipped, 0 undefined
Took 0m1.961s
```

Expected Result

The feature is executed three times, once per Examples row. A successful run reports one passed feature, three passed scenarios, and all steps passed. The test requires an internet connection and an installed Chrome browser; Selenium Manager obtains the compatible driver automatically.

Conclusion

Scenario Outline is a practical BDD pattern for data-driven testing. It reduces duplicated steps and makes coverage easy to expand: adding another credential combination requires only one new row in the Examples table.